Java exception handling

What exceptions are, how they differ from errors, the checked v/s unchecked split and the class hierarchy, how try-catch-finally and try-with-resources work, throw v/s throws, and custom exceptions. Definitions stay tight; the answer does the teaching.

Contents

1. What is an exception?

An exception is an **unexpected event during program execution that disrupts the normal flow**. In Java, exceptions are objects that can be thrown and caught.

```
int a = 10;
int b = 0;
int result = a / b;
```

Output:

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
```

Dividing by zero raises an exception, and normal execution stops right there.

Why we need exceptions: without handling, one error takes the whole program down.

```
System.out.println("Step 1");
int result = 10 / 0;
System.out.println("Step 2");
```

Output:

```
Step 1
Exception in thread "main"
java.lang.ArithmeticException: / by zero
```

Step 2 never runs, because the exception terminates the program. With handling, we catch the error and carry on:

```java
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero.");
}

System.out.println("Program continues...");
```

Output:

```
Cannot divide by zero.
Program continues...
```

The program deals with the error and keeps going.

Common exceptions:

Exception	Cause
`ArithmeticException`	Divide by zero
`NullPointerException`	Accessing a `null` reference
`ArrayIndexOutOfBoundsException`	Invalid array index
`StringIndexOutOfBoundsException`	Invalid string index
`NumberFormatException`	Invalid number conversion
`ClassCastException`	Invalid type casting
`IOException`	File or I/O operation failure

2. Difference between Error and Exception.

An **exception** is a condition during execution that the application can usually handle or recover from. An **error** is a serious problem, generally from the JVM or the environment, that application code should not normally try to handle.

Feature	Exception	Error
Meaning	A recoverable problem	A serious JVM or system problem
Can be handled	Yes	Generally no
Caused by	Application logic, external resources, user input	The JVM, system resources, or environment
Recovery	Usually possible	Usually not possible
Inheritance	Extends `Exception`	Extends `Error`
Examples	`IOException`, `SQLException`, `NullPointerException`	`OutOfMemoryError`, `StackOverflowError`

On the exception side, the catch-and-continue from question 1 is the typical shape: the application catches `ArithmeticException` and keeps running.

An error is different. Here infinite recursion exhausts the stack:

```
public class Main {
    public static void main(String[] args) {
        main(args);
    }
}
```

Output:

```
java.lang.StackOverflowError
```

The JVM throws an `Error`, and the application cannot realistically recover from it. That is the line: exceptions are for problems we can plan around, errors are the ones we usually cannot.

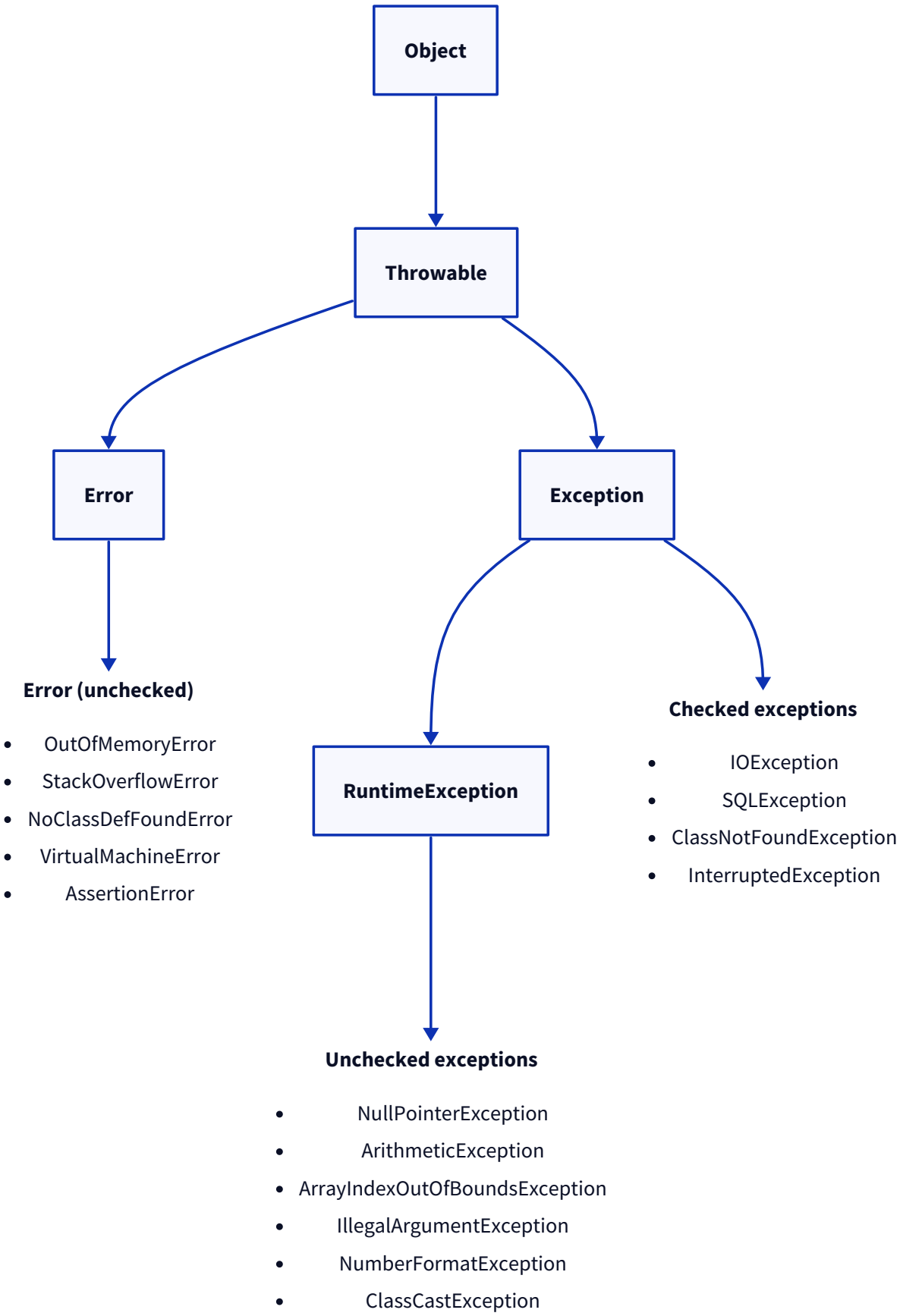
3. Difference between checked and unchecked exceptions.

Checked exceptions are verified by the compiler: they must be handled with try-catch or declared with `throws`. **Unchecked** exceptions are not checked at compile time and usually point to programming mistakes that should be fixed rather than caught.

Feature	Checked exception	Unchecked exception
Checked by compiler	Yes	No
Must handle or declare	Yes	No
Inheritance	Extends `Exception` (but not `RuntimeException`)	Extends `RuntimeException`
Cause	External conditions (file, network, DB)	Programming mistakes
Recovery	Usually possible	Usually by fixing the code
Examples	`IOException`, `SQLException`, `ClassNotFoundException`	`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`

4. Error and exception hierarchy.

Everything throwable descends from `Object` through `Throwable`, which splits into `Error` and `Exception`. Under `Exception`, the checked exceptions sit directly, while `RuntimeException` and everything under it are unchecked.

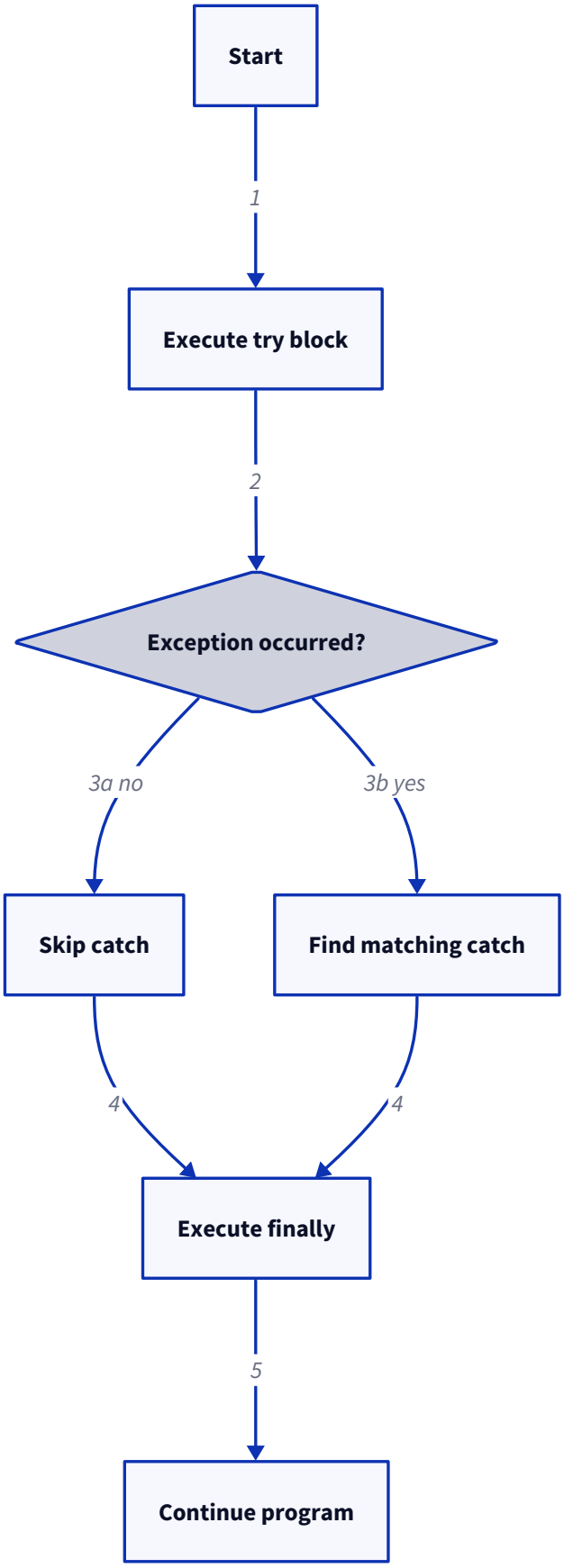


5. How does try-catch-finally work?

The `try` block holds code that might throw, `catch` handles the exception, and `finally` runs either way. That last part is the key: `finally` executes whether or not an exception was thrown, which is why cleanup belongs there.

```
try {
    // code that may throw an exception
} catch (ExceptionType e) {
    // handle the exception
} finally {
    // cleanup code
}
```

Flow of execution:



Scenario 1, no exception: `try` runs, `catch` is skipped, `finally` still runs.

```java
try {
    System.out.println("Inside try");
} catch (Exception e) {
    System.out.println("Inside catch");
} finally {
    System.out.println("Inside finally");
}
```

Output:

```
Inside try
Inside finally
```

Scenario 2, exception occurs: the exception is thrown, the JVM finds the matching `catch`, then `finally` runs.

```java
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Cleanup");
}
```

Output:

```
Cannot divide by zero
Cleanup
```

Scenario 3, exception not caught: no `catch` matches, `finally` still runs, then the exception propagates to the caller (or ends the program if nothing handles it).

```java
try {
    int[] arr = new int[2];
    System.out.println(arr[5]);
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Error");
} finally {
    System.out.println("Cleanup");
}
```

Output:

```
Cleanup
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException
```

Note that `finally` ran before the exception propagated. That is its whole point.

Multiple catch blocks: the JVM runs only the first `catch` that matches.

```java
try {
    // code
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Error");
} catch (NullPointerException e) {
    System.out.println("Null Pointer");
} catch (Exception e) {
    System.out.println("General Exception");
}
```

Order of catch blocks: specific exceptions must come before general ones.

Correct, specific first:

```java
try {
    // code
} catch (ArithmeticException e) {
} catch (Exception e) {
}
```

Incorrect, general first:

```java
try {
    // code
} catch (Exception e) {
} catch (ArithmeticException e) {
}
```

This fails to compile with `Unreachable catch block for ArithmeticException`, because `Exception` already catches everything, so the `ArithmeticException` block can never be reached.

6. When is finally not executed?

`finally` runs in almost every case. The exceptions are the situations where the JVM or the process dies before control ever reaches it.

Situation	Does `finally` run?
`System.exit()` is called	No
The JVM crashes	No
The process is forcibly killed (`kill -9`)	No
Power failure or OS crash	No
An infinite loop before `finally`	No, it is never reached

The common thread is that `finally` is a Java-level guarantee, so it only holds while the JVM is alive and execution actually gets there. Anything that stops the JVM itself, or never reaches the block, skips it.

7. Difference between throw and throws.

`throw` explicitly throws an exception from inside a method. `throws` goes in a method signature to declare that the method may throw one or more exceptions, handing the responsibility to the caller.

Feature	`throw`	`throws`
Purpose	Explicitly throws an exception	Declares that a method may throw exceptions
Used with	An exception object	Exception class(es)
Location	Inside a method	In the method signature
Number	One exception object at a time	One or more exception types
Responsibility	Throws the exception immediately	Delegates handling to the caller

throw: raise an exception manually.

```
public class Main {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new IllegalArgumentException("Age must be at least 18");
        }
    }
}
```

Output:

```
Exception in thread "main"
java.lang.IllegalArgumentException: Age must be at least 18
```

throws: declare that a method may throw, so the caller must handle or declare it.

```
import java.io.IOException;

public class Main {
    static void readFile() throws IOException {
        throw new IOException("File not found");
    }

    public static void main(String[] args) throws IOException {
        readFile();
    }
}
```

Using both together: `throw` creates and raises the exception, and `throws` declares that the method may throw it.

```java
import java.io.IOException;

class FileService {
    void read() throws IOException {
        throw new IOException("File missing");
    }
}
```

8. How do you create a custom exception?

A custom exception is a user-defined exception, made by extending `Exception` (checked) or `RuntimeException` (unchecked). It lets us represent business-specific error conditions.

Checked custom exception: extend `Exception`.

```java
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Using it, the caller has to handle or declare it:

```java
public class Main {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be at least 18");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(16);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Age must be at least 18
```

Unchecked custom exception: extend `RuntimeException`.

```java
class InvalidAmountException extends RuntimeException {
    public InvalidAmountException(String message) {
        super(message);
    }
}
```

```
public class Main {
    static void withdraw(double amount) {
        if (amount <= 0) {
            throw new InvalidAmountException("Amount must be positive");
        }
    }
}
```

No `throws` clause or try-catch is required here.

9. Should custom exceptions extend Exception or RuntimeException?

It depends on whether the caller can reasonably recover from the problem.

- Extend `Exception` for recoverable business conditions, where the caller is expected to handle the failure.
- Extend `RuntimeException` for programming errors, invalid API usage, or unrecoverable conditions.

The rule of thumb: if the caller has a sensible action to take when the failure happens, make it checked so the compiler reminds them to deal with it. If it only signals a bug, make it unchecked so it does not clutter every method signature on the way up.

10. What is try-with-resources, and which interfaces support it?

Try-with-resources, added in Java 7, **closes resources automatically after use**, so we no longer have to close them by hand in a finally block.

Why it was introduced: before Java 7, closing was manual.

```
FileInputStream file = null;
try {
    file = new FileInputStream("data.txt");
    // read file
} finally {
    if (file != null) {
        file.close();
    }
}
```

That is a lot of boilerplate, it is easy to forget the close, it can leak resources, and `close()` can itself throw. Try-with-resources fixes all of that.

Syntax: declare the resource in the `try(...)` header.

```
try (Resource resource = new Resource()) {
    // use resource
}
```

The resource is closed automatically, even if an exception is thrown.

```java
import java.io.FileInputStream;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try (FileInputStream file = new FileInputStream("data.txt")) {
            System.out.println(file.read());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

No explicit `close()` is needed.

Which interfaces support it: the resource must implement one of these.

Interface	Introduced	Description
`AutoCloseable`	Java 7	The base interface for auto-closable resources
`Closeable`	Java 5	Extends `AutoCloseable`, mainly for I/O resources

`Closeable` extends `AutoCloseable`, so every `Closeable` is an `AutoCloseable`, but not the other way round.

Common classes that support it:

Class	Interface
`FileInputStream`	`Closeable`
`FileOutputStream`	`Closeable`
`BufferedReader`	`Closeable`
`BufferedWriter`	`Closeable`
`FileReader`	`Closeable`
`FileWriter`	`Closeable`
`Scanner`	`Closeable`
`Socket`	`Closeable`
`Connection` (JDBC)	`AutoCloseable`
`Statement`	`AutoCloseable`
`PreparedStatement`	`AutoCloseable`
`ResultSet`	`AutoCloseable`

11. If unchecked exceptions are more convenient, why do checked exceptions still exist?

Because some failures are **expected and recoverable**. Checked exceptions force the caller to acknowledge and handle the cases where recovery is realistic, whereas unchecked exceptions are meant for programming errors.

Why not make everything unchecked: say we are reading a file.

```
FileReader reader = new FileReader("data.txt");
```

The file might not exist, might have the wrong permissions, or might be locked by another process. None of those are programming bugs, they are external problems the caller may well be able to recover from, for example by asking the user to pick another file, retrying later, or falling back to a default configuration.

That is exactly why Java makes `IOException` checked: it nudges the caller to plan for a failure that is likely and handleable, rather than letting it slip through as an afterthought.